Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования

«Казанский (Приволжский) федеральный университет»

Институт вычислительной математики и информационных технологий

Кафедра прикладной математики и искусственного интеллекта

ПРОЕКТ

Многослойный персептрон

Выполнил:

студент гр. 09-322

Тляумбетов Фархат Салаватович

Казань 2026

СОДЕРЖАНИЕ

1 Введение 3

2 Исследование и подготовка данных 4

3 Математическая модель и реализация 7

4 Эксперименты и оценка модели 8

4.1 Сравнение вариантов 8

4.2 Оценка выбранной модели 9

5 Результаты и выводы 10

Приложение А. Листинг программы 11

1 Введение

В работе рассматриваются признаки ядер клеток молочной железы, полученные при исследовании биопсийного материала. Каждому объекту соответствует один из двух классов: B (доброкачественный) или M (злокачественный). По измеренным характеристикам требуется предсказать класс объекта, который не участвовал в расчёте градиентов при обучении.

Входом модели служит вектор из 30 числовых признаков. На выходе сеть должна выдавать вероятности двух классов, сумма которых равна единице. Предсказанием считается класс с большей вероятностью. Идентификатор объекта и известный диагноз не используются как входные признаки.

В работе используется сеть с двумя скрытыми слоями. Расчёт прогноза, обратное распространение ошибки и обновление весов реализованы самостоятельно. Помимо анализа данных, исследуются функции активации и влияние удаления признаков на качество классификации.

Качество оценивается по бинарной кросс-энтропии BCE, доле правильных ответов accuracy и полноте recall класса M. Последняя метрика показывает, какую долю объектов M удалось обнаружить. Отдельно учитываются ошибки, при которых объект M отнесён к B.

2 Исследование и подготовка данных

Набор данных содержит 569 объектов. Каждая строка data.csv включает идентификатор ID, диагноз и 30 числовых признаков. Из-за отсутствия заголовка использованы имена от feature_1 до feature_30. ID исключён из входов, метки закодированы как B=0 и M=1. Соотношение классов показано на рисунке 1.

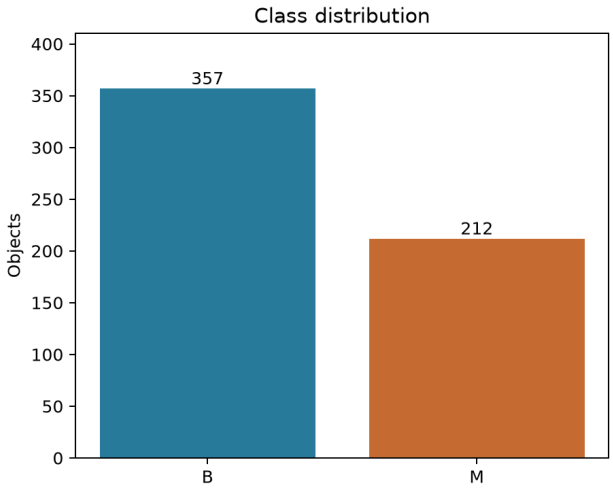


Рисунок 1: Распределение объектов по классам

Классу B принадлежат 357 объектов (62,74%), классу M принадлежат 212 (37,26%). Доли классов сохраняются при разделении выборки. Пропусков, бесконечностей, повторяющихся ID, одинаковых векторов признаков и постоянных столбцов нет. Поэтому заполнение пропусков и удаление дубликатов не выполнялись.

Для всех признаков рассчитаны среднее, стандартное отклонение, минимум и максимум. Масштабы различаются: например, feature_4 изменяется от 143,5 до 2501, а feature_5 изменяется от 0,05263 до 0,1634. Это обосновывает стандартизацию перед обучением.

Распределения всех 30 признаков представлены на рисунке 2. Границы интервалов общие для обоих классов, площадь каждой гистограммы равна единице.

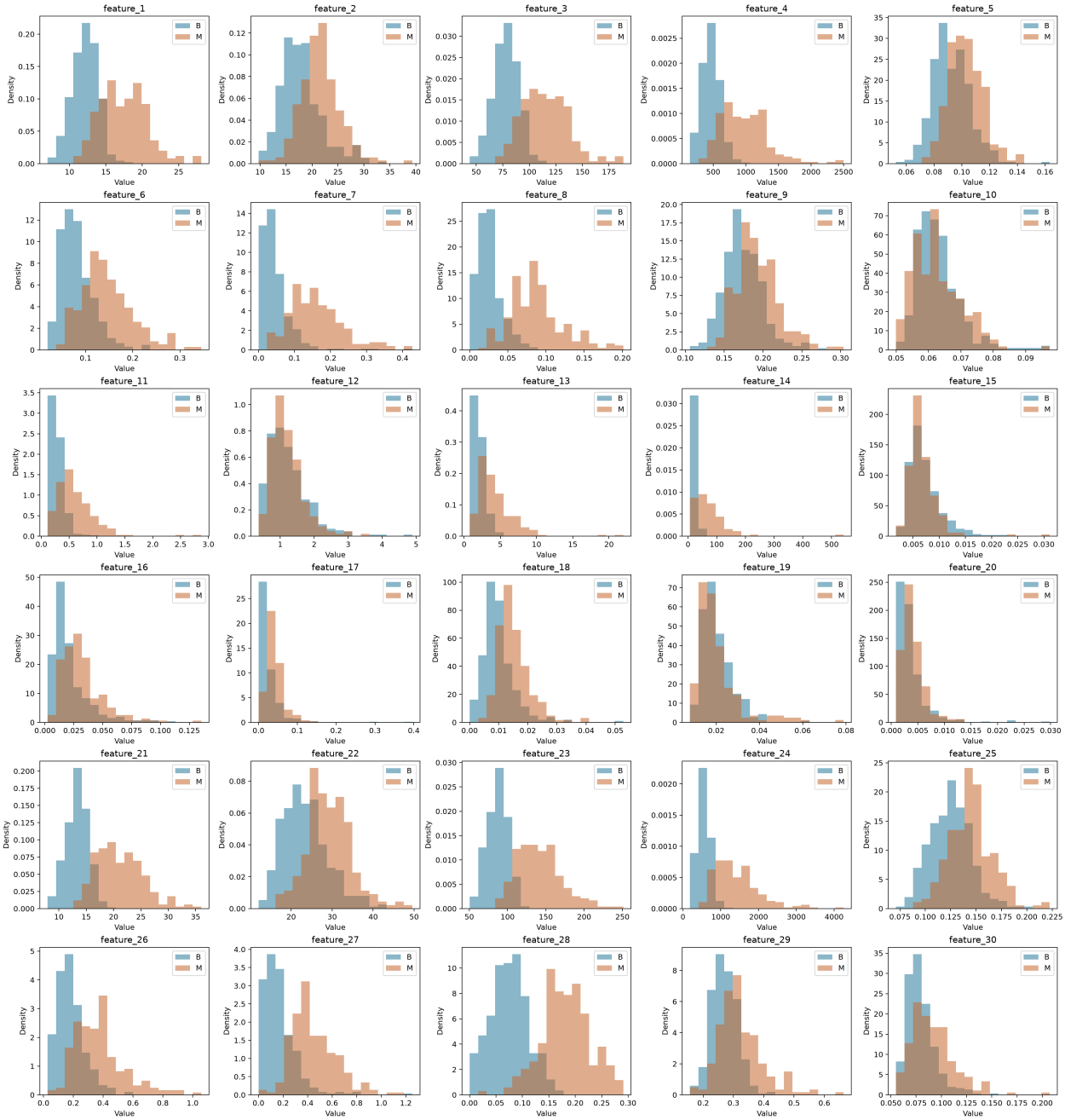


Рисунок 2: Распределения признаков для классов B и M

Для части признаков распределения B и M заметно различаются. Их перекрытие не означает бесполезности признака: сеть использует признаки совместно. Это предположение дополнительно проверяется экспериментами с удалением столбцов.

Матрица корреляций на рисунке 3 показывает группы взаимосвязанных признаков. Например, корреляция feature_1 и feature_3 равна 0,9979. Корреляция сама по себе не доказывает, что удаление одного признака улучшит прогноз.

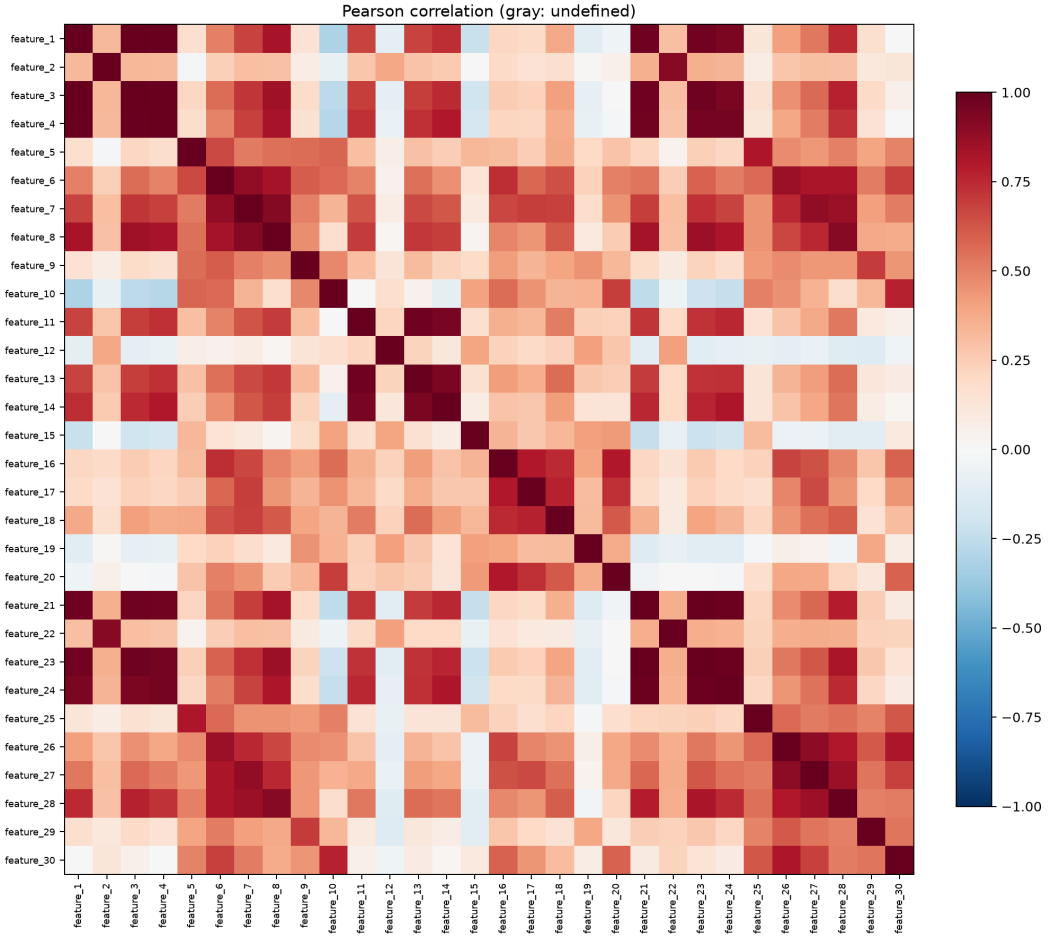


Рисунок 3: Корреляции Пирсона между признаками

Исходные данные разделены один раз внутри каждого класса с seed=42: обучающая часть содержит 456 объектов (286 B и 170 M), проверочная часть содержит 113 объектов (71 B и 42 M). Части не пересекаются и сохраняются в отдельные CSV; доля validation составляет примерно 20% из-за округления по классам.

Стандартизация рассчитывается только по train:

$$\widetilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}, \qquad \mu_j = \text{mean}(x_j), \quad \sigma_j = \text{std}(x_j). \tag{1}$$

Для validation и прогнозов применяются те же параметры. При нулевом стандартном отклонении делитель заменяется единицей.

3 Математическая модель и реализация

Сеть содержит 30 входов, два скрытых слоя по 24 нейрона с sigmoid и два выхода с softmax; всего 1394 обучаемых параметра. Для пакета из m объектов $A_0 = \widetilde{X}$; прямой проход скрытого слоя имеет вид

$$Z_l = A_{l-1}W_l + b_l, \qquad A_l = \sigma(Z_l), \qquad \sigma(z) = \frac{1}{1+e^{-z}}. \tag{2}$$

На выходе логиты $Z_3 = A_2W_3 + b_3$ преобразуются в вероятности классов:

$$p_{ik} = \frac{\exp(z_{ik} - c_i)}{\sum_{j=0}^{1}\exp(z_{ij} - c_i)}, \qquad c_i = \max_j z_{ij}, \qquad \widehat{y}_i = \arg\max_k p_{ik}. \tag{3}$$

Сдвиг предотвращает переполнение экспоненты. Бинарная кросс-энтропия вычисляется через устойчивые логарифмы вероятностей:

$$L = -\frac{1}{m}\sum_{i=1}^{m}\big[(1-y_i)\log p_{i0} + y_i \log p_{i1}\big]. \tag{4}$$

Для единичных векторов меток Y правило цепочки даёт:

$$\begin{aligned} D_3 &= (P-Y)/m, \\ \nabla_{W_l}L &= A_{l-1}^T D_l, \qquad \nabla_{b_l}L = \sum_i D_{l,i:}, \\ D_{l-1} &= (D_l W_l^T)\odot A_{l-1}\odot(1-A_{l-1}), \quad l>1. \end{aligned} \tag{5}$$

Все градиенты рассчитываются до обновления параметров:

$$W_l^{t+1} = W_l^t - \eta\nabla_{W_l}L, \qquad b_l^{t+1} = b_l^t - \eta\nabla_{b_l}L. \tag{6}$$

Начальные веса задаются методом Xavier Uniform, смещения равны нулю:

$$W_{ij} \sim U(-a,a), \qquad a = \sqrt{\frac{6}{n_{in}+n_{out}}}. \tag{7}$$

Xavier ограничивает масштаб весов с учётом числа входов n_{in} и выходов n_{out} слоя. Случайность нарушает симметрию нейронов. Это простой вариант для нашей сети с sigmoid, но не гарантия от затухания градиентов.

4 Эксперименты и оценка модели

4.1 Сравнение вариантов

Выборки и скрытые слои не изменялись. Обучение выполнялось 100 эпох с шагом 0,01, пакетом из 8 объектов и Xavier Uniform. Seed=0 выбран по предварительной оценке на проверочной выборке. Строки train перемешивались перед каждой эпохой; сохранялась модель с минимальной проверочной BCE.

Для sigmoid, tanh и ReLU на всех 30 признаках сохранены эпохи 100, 27 и 43 соответственно. Удаление признаков проверялось с sigmoid; корреляции для отбора рассчитывались только по train.

Связанные признаки отбирались последовательно в исходном порядке: столбец удалялся при $|r| > 0{,}95$ хотя бы с одним уже оставленным. Удалены номера 3, 4, 13, 14, 21 и 23. Слабыми условно названы признаки с $|r(x_j, y)| < 0{,}1$, где B=0 и M=1: это номера 10, 12, 15, 19 и 20. Так измеряется только линейная связь, а не вся информация о классе.

Отдельно проверены удаление feature_2 и совместное применение обоих правил отбора. Результаты показаны в таблице 1. Для всех вариантов sigmoid сохранена эпоха 100; FN означает число пропущенных M.

Таблица 1: Сравнение активаций и наборов признаков

Вариант	Входов	BCE	Accuracy, %	Recall M, %	FN
Sigmoid	30	0,081849	98,23	97,62	1
Tanh	30	0,075670	96,46	97,62	1
ReLU	30	0,070793	96,46	97,62	1
Без связанных	24	0,079672	97,35	95,24	2
Без слабых	25	0,088273	97,35	97,62	1
Без feature_2	29	0,086687	97,35	97,62	1
Оба правила	19	0,093284	96,46	92,86	3

У tanh и ReLU BCE ниже, но ошибок четыре вместо двух: потери учитывают вероятности, а accuracy учитывает метки. Все варианты удаления снизили accuracy; совместный отбор увеличил число пропущенных M с одного до трёх.

Для ReLU существует инициализация He, учитывающая обнуление отрицательных входов. Здесь Xavier одинаков для всех активаций, поэтому сравнение не определяет лучшую активацию при индивидуальном подборе настроек.

4.2 Оценка выбранной модели

Основной вариант использует sigmoid, все 30 признаков и seed=0. Сохранена модель эпохи 100. Её показатели приведены в таблице 2, кривые обучения показаны на рисунке 4.

Таблица 2: Показатели выбранной модели

Показатель	Обучение	Проверка
BCE	0,064815	0,081849
Accuracy, %	98,6842	98,2301
Правильные ответы	450 / 456	111 / 113

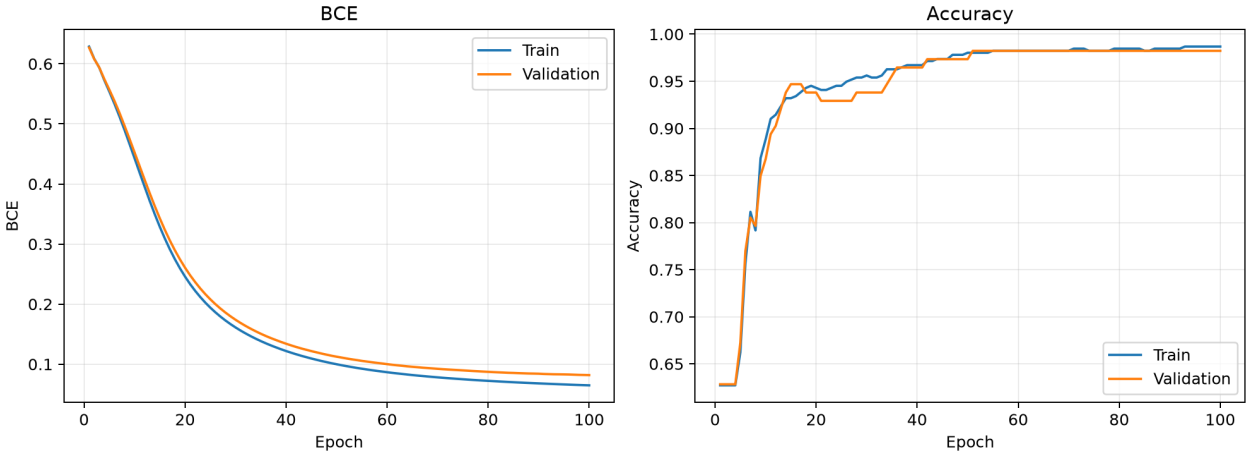


Рисунок 4: Изменение BCE и accuracy при обучении

Матрица ошибок: строки соответствуют истинным классам, столбцы соответствуют прогнозам.

	$\widehat{B}$	$\widehat{M}$
B	70	1
M	1	41

$$\text{Recall}_M = \frac{TP}{TP + FN} = \frac{41}{42} = 97{,}62\%. \tag{8}$$

Получены один ложный прогноз M и один пропущенный M. При приоритете обнаружения M важны recall и число пропусков, а не только accuracy.

Правильность обратного прохода дополнительно проверена численным дифференцированием: максимальное абсолютное расхождение градиентов в контрольном примере составило менее $1{,}7 \cdot 10^{-11}$.

5 Результаты и выводы

Реализованный многослойный персептрон правильно классифицировал 111 из 113 объектов проверочной выборки. Доля правильных ответов составила 98,23%, полнота класса M составила 97,62%. Допущены две ошибки: один объект B отнесён к M и один объект M пропущен. Дополнительно к общей точности оценивалась полнота обнаружения M.

Анализ показал различие масштабов признаков, которое учтено стандартизацией по обучающим данным. Высокая корреляция между признаками и слабая линейная связь с классом не стали достаточным основанием для удаления: во всех проверенных вариантах оно уменьшило долю правильных ответов. В результате оставлены все 30 признаков и sigmoid. При тех же настройках tanh и ReLU дали меньшую кросс-энтропию, но больше ошибок классификации.

Полученные показатели характеризуют выбранное разбиение и проверенные настройки. Проверочная выборка использовалась при выборе seed, варианта модели и сохраняемой эпохи, поэтому результат нельзя считать независимой оценкой качества. Для оценки работы на новых объектах потребуется отдельная тестовая выборка, не участвующая в подборе настроек.

ПРИЛОЖЕНИЕ А

Листинг программы

Реализация проверена на Python 3.11. Используются NumPy для вычислений и Matplotlib для построения графиков. Файл model.npz хранит веса, смещения, архитектуру и параметры стандартизации; команда predict загружает готовую модель. Команды запуска из корня проекта:

```
python3.11 src/mlp.py analyze
python3.11 src/mlp.py split
python3.11 src/mlp.py train --seed 0
python3.11 src/mlp.py predict
```

Исходный файл src/mlp.py:

```
import argparse
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

SCRIPT_DIR = Path(__file__).resolve().parent
ROOT = SCRIPT_DIR.parent if SCRIPT_DIR.name == "src" else SCRIPT_DIR
DATA = ROOT / "task" / "data.csv"
RESULTS = ROOT / "results"
SPLIT = RESULTS / "split"
SEED = 42

# 1. Загрузка данных

def load_data(path):
    rows = np.loadtxt(path, delimiter=",", dtype=str, ndmin=2)
    x = rows[:, 2:].astype(float)
    y = (rows[:, 1] == "M").astype(int)

    return rows, x, y

# 2. Анализ данных

def analyze(rows, x, y):
    names = [f"feature_{i + 1}" for i in range(x.shape[1])]
    counts = np.bincount(y, minlength=2)
    output = RESULTS / "analysis"
    output.mkdir(parents=True, exist_ok=True)

    print(f"Объектов: {len(x)}, признаков: {x.shape[1]}")
    print(f"B: {counts[0]}, M: {counts[1]}")
    print(f"Повторяющихся ID: {len(rows) - len(np.unique(rows[:, 0]))}")
    print(f"Повторяющихся наборов признаков: {len(x) - len(np.unique(x, axis=0))}"
    )
    print(f"Постоянных признаков: {np.sum(x.std(axis=0) == 0)}")
    print(f"Пропусков и бесконечных значений: {np.sum(~np.isfinite(x))}")
    print(f"Признак{'':<14} Среднее{'':>10} Ст{'. откл.':>10} Мин{'.':>10}
    Макс{'.':>10}")

    statistics = np.column_stack(
        [x.mean(axis=0), x.std(axis=0, ddof=1), x.min(axis=0), x.max(axis=0)]
    )

    for name, values in zip(names, statistics):
        print(f"{name:<14}" + "".join(f"{value:10.5g} " for value in values))
```

```
    fig, ax = plt.subplots(figsize=(5, 4), layout="constrained")
    bars = ax.bar(["B", "M"], counts, color=["#287a9b", "#c56b32"])
    ax.bar_label(bars)
    ax.set(title="Class distribution", ylabel="Objects", ylim=(0, counts.max() *
    1.15))
    fig.savefig(output / "classes.png", dpi=180)
    plt.close(fig)

    fig, axes = plt.subplots(6, 5, figsize=(20, 21), layout="constrained")

    for i, ax in enumerate(axes.flat):
        bins = np.histogram_bin_edges(x[:, i], bins=18)

        for label, color in enumerate(("#287a9b", "#c56b32")):
            ax.hist(
                x[y == label, i], bins=bins, density=True, alpha=0.55,
                label=["B", "M"][label], color=color,
            )

        ax.set(title=names[i], xlabel="Value", ylabel="Density")
        ax.legend()

    fig.savefig(output / "distributions.png", dpi=180)
    plt.close(fig)

    with np.errstate(divide="ignore", invalid="ignore"):
        correlations = np.corrcoef(x, rowvar=False)

    fig, ax = plt.subplots(figsize=(11, 10), layout="constrained")
    cmap = plt.get_cmap("RdBu_r").with_extremes(bad="lightgray")
    image = ax.imshow(np.ma.masked_invalid(correlations), cmap=cmap, vmin=-1, vmax
    =1)
    ax.set_xticks(range(len(names)), names, rotation=90, fontsize=7)
    ax.set_yticks(range(len(names)), names, fontsize=7)
    ax.set_title("Pearson correlation (gray: undefined)")
    fig.colorbar(image, ax=ax, shrink=0.8)
    fig.savefig(output / "correlations.png", dpi=180)
    plt.close(fig)

# 3. Разделение и стандартизация

def split_data(y):
    rng = np.random.default_rng(SEED)
    train, valid = [], []

    for label in (0, 1):
        indices = rng.permutation(np.flatnonzero(y == label))

        if len(indices) < 2:
            raise ValueError("Для разделения нужны минимум два объекта каждого
    класса")

        count = max(1, round(len(indices) * 0.2))

        valid.extend(indices[:count])
        train.extend(indices[count:])

    return rng.permutation(train), rng.permutation(valid)

def save_split(rows, y):
    training, validation = split_data(y)
    SPLIT.mkdir(parents=True, exist_ok=True)

    for name, indices in (("train", training), ("valid", validation)):
        np.savetxt(
            SPLIT / f"{name}.csv", rows[indices], delimiter=",", fmt="%s",
        )
        counts = np.bincount(y[indices], minlength=2)
```

```
        print(f"{name}: {len(indices)} объектов, B={counts[0]}, M={counts[1]}")

    print(f"Выборки сохранены в {SPLIT}")

def standardize(x, mean, std):
    return (x - mean) / std

# 4. Функции активации и их производные

def relu(z):
    return np.maximum(0, z)

def relu_derivative(a):
    return (a > 0).astype(float)

def sigmoid(z):
    exp_z = np.exp(-np.abs(z))
    return np.where(z >= 0, 1 / (1 + exp_z), exp_z / (1 + exp_z))

def sigmoid_derivative(a):
    return a * (1 - a)

def tanh(z):
    return 2 * sigmoid(2 * z) - 1

def tanh_derivative(a):
    return 1 - a ** 2

def softmax(z):
    shifted = z - z.max(axis=1, keepdims=True)
    log_p = shifted - np.log(np.exp(shifted).sum(axis=1, keepdims=True))
    p = np.exp(log_p)

    return p, log_p

ACTIVATIONS = {
    "relu": (relu, relu_derivative),
    "sigmoid": (sigmoid, sigmoid_derivative),
    "tanh": (tanh, tanh_derivative),
}

# 5. Инициализация весов и прямой проход

def initialize(layers, seed=SEED):
    rng = np.random.default_rng(seed)
    weights, biases = [], []

    for n_in, n_out in zip(layers[:-1], layers[1:]):
        limit = np.sqrt(6 / (n_in + n_out))
        weights.append(rng.uniform(-limit, limit, size=(n_in, n_out)))
        biases.append(np.zeros(n_out))

    return weights, biases

def forward(x, weights, biases, activation="sigmoid"):
    activate, _ = ACTIVATIONS[activation]
    activations = [x]

    for weight, bias in zip(weights[:-1], biases[:-1]):
        z = activations[-1] @ weight + bias
        activations.append(activate(z))

    logits = activations[-1] @ weights[-1] + biases[-1]
    p, log_p = softmax(logits)

    return activations, p, log_p

# 6. Функция потерь, метрики и градиент выхода
```

```
def binary_cross_entropy(y, log_p):
    return -np.mean((1 - y) * log_p[:, 0] + y * log_p[:, 1])

def accuracy(y, p):
    return np.mean(p.argmax(axis=1) == y)

def metrics(y, p, log_p):
    return binary_cross_entropy(y, log_p), accuracy(y, p)

def softmax_bce_gradient(p, y):
    delta = p.copy()
    delta[np.arange(len(y)), y] -= 1

    return delta / len(y)

# 7. Обратное распространение и градиентный спуск

def backward(activations, p, y, weights, activation="sigmoid"):
    _, derivative = ACTIVATIONS[activation]
    delta = softmax_bce_gradient(p, y)
    dw, db = [None] * len(weights), [None] * len(weights)

    for i in range(len(weights) - 1, -1, -1):
        dw[i] = activations[i].T @ delta
        db[i] = delta.sum(axis=0)

        if i > 0:
            delta = (delta @ weights[i].T) * derivative(activations[i])

    return dw, db

def gradient_descent(weights, biases, dw, db, learning_rate):
    for i in range(len(weights)):
        weights[i] -= learning_rate * dw[i]
        biases[i] -= learning_rate * db[i]

# 8. Обучение, сохранение модели и графики

def train(x_train, y_train, x_valid, y_valid, args):
    mean = x_train.mean(axis=0)
    std = x_train.std(axis=0)
    std[std == 0] = 1
    x_train = standardize(x_train, mean, std)
    x_valid = standardize(x_valid, mean, std)

    layers = [x_train.shape[1], *args.hidden, 2]
    weights, biases = initialize(layers, args.seed)
    rng = np.random.default_rng(args.seed)
    history = []
    best_loss = np.inf

    for epoch in range(1, args.epochs + 1):
        order = rng.permutation(len(x_train))

        for start in range(0, len(order), args.batch_size):
            batch = order[start:start + args.batch_size]
            activations, p, _ = forward(x_train[batch], weights, biases, args.
    activation)
            dw, db = backward(activations, p, y_train[batch], weights, args.
    activation)

            gradient_descent(weights, biases, dw, db, args.lr)

        _, p, log_p = forward(x_train, weights, biases, args.activation)
        train_loss, train_accuracy = metrics(y_train, p, log_p)
        _, p, log_p = forward(x_valid, weights, biases, args.activation)
        valid_loss, valid_accuracy = metrics(y_valid, p, log_p)

        if not np.isfinite([train_loss, valid_loss]).all():
```

```
            raise FloatingPointError("Потери обучения стали NaN или бесконечными")

        if valid_loss < best_loss:
            best_loss, best_epoch = valid_loss, epoch
            best_weights = [weight.copy() for weight in weights]
            best_biases = [bias.copy() for bias in biases]

        history.append([train_loss, valid_loss, train_accuracy, valid_accuracy])

        print(
            f"Эпоха {epoch:3}/{args.epochs}: "
            f"BCE={train_loss:.6f}, val_BCE={valid_loss:.6f}, "
            f"accuracy={train_accuracy:.2%}, val_accuracy={valid_accuracy:.2%}"
        )

    RESULTS.mkdir(exist_ok=True)
    params = {}

    for i in range(len(weights)):
        params[f"w{i}"] = best_weights[i]
        params[f"b{i}"] = best_biases[i]

    np.savez(
        RESULTS / "model.npz", layers=layers, mean=mean, std=std,
        epoch=best_epoch, activation=args.activation, seed=args.seed, **params,
    )
    print(f"Сохранена эпоха {best_epoch} с минимальной validation BCE={best_loss
    :.6f}")

    history = np.array(history)
    epochs = np.arange(1, args.epochs + 1)
    fig, axes = plt.subplots(1, 2, figsize=(11, 4), layout="constrained")

    for ax, column, title in zip(axes, (0, 2), ("BCE", "Accuracy")):
        ax.plot(epochs, history[:, column], label="Train")
        ax.plot(epochs, history[:, column + 1], label="Validation")
        ax.set(xlabel="Epoch", ylabel=title, title=title)
        ax.legend()
        ax.grid(alpha=0.25)

    fig.savefig(RESULTS / "learning_curves.png", dpi=180)
    if not args.no_show:
        plt.show()
    plt.close(fig)

# 9. Прогнозирование и оценка

def predict(x, y):
    with np.load(RESULTS / "model.npz", allow_pickle=False) as model:
        weights = [model[f"w{i}"] for i in range(len(model["layers"]) - 1)]
        biases = [model[f"b{i}"] for i in range(len(weights))]
        activation = str(model["activation"])
        x = standardize(x, model["mean"], model["std"])

    _, p, log_p = forward(x, weights, biases, activation)
    loss, acc = metrics(y, p, log_p)
    matrix = np.zeros((2, 2), dtype=int)
    np.add.at(matrix, (y, p.argmax(axis=1)), 1)
    print(f"Объектов: {len(y)}; BCE={loss:.6f}; accuracy={acc:.4%}")

    if matrix[1].sum():
        recall = matrix[1, 1] / matrix[1].sum()
        print(f"Recall класса M: {recall:.4%}")
    else:
        print("Recall класса M: не определён в( выборке нет M)")

    print("Матрица ошибок: строки — истинные B/M, столбцы — прогноз B/M")
    print(matrix)
```

```
# 10. Запуск этапов пайплайна

def positive_int(value):
    number = int(value)

    if number < 1:
        raise argparse.ArgumentTypeError("Нужно целое число больше нуля")

    return number

def positive_float(value):
    number = float(value)

    if not np.isfinite(number) or number <= 0:
        raise argparse.ArgumentTypeError("Нужно конечное число больше нуля")

    return number

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("command", choices=["analyze", "split", "train", "predict"
    ])
    parser.add_argument("data", nargs="?", type=Path)
    parser.add_argument("--valid-data", type=Path, default=SPLIT / "valid.csv")
    parser.add_argument("--hidden", "--layer", type=positive_int, nargs="+",
    default=[24, 24])
    parser.add_argument("--activation", choices=ACTIVATIONS, default="sigmoid")
    parser.add_argument("--epochs", type=positive_int, default=100)
    parser.add_argument("--lr", "--learning_rate", type=positive_float, default
    =0.01)
    parser.add_argument("--batch-size", "--batch_size", type=positive_int, default
    =8)
    parser.add_argument("--seed", type=int, default=SEED)
    parser.add_argument("--no-show", action="store_true", help="Сохранить графики
    без открытия окна")
    args = parser.parse_args()

    if args.seed < 0:
        parser.error("Seed должен быть целым неотрицательным числом")

    default_data = {
        "analyze": DATA,
        "split": DATA,
        "train": SPLIT / "train.csv",
        "predict": SPLIT / "valid.csv",
    }
    rows, x, y = load_data(args.data or default_data[args.command])

    if args.command == "analyze":
        analyze(rows, x, y)
    elif args.command == "split":
        try:
            save_split(rows, y)
        except ValueError as error:
            parser.error(str(error))
    elif args.command == "train":
        _, x_valid, y_valid = load_data(args.valid_data)

        try:
            with np.errstate(over="raise", invalid="raise", divide="raise"):
                train(x, y, x_valid, y_valid, args)
        except FloatingPointError:
            parser.error("Ошибка численных вычислений при обучении. Уменьшите --lr
    ")
    else:
        predict(x, y)

if __name__ == "__main__":
    main()
```